SUBSYSTEM 2 · PHASE 1 · COMPLETED

Stochastic Simulation Engine

LogiTwin AI Engine — Subsystem 2 Complete Documentation

Disruption detection · Graph traversal · Impact scoring · Neo4j event writes

7	5	21	3
New Files	New Endpoints	Tests Passing	Live Simulations

Celery 5.4 · Redis 7 · asyncio · Neo4j Cypher · pytest-asyncio · BFS Traversal · Impact Scorer

01 — What Subsystem 2 Added to LogiTwin

Subsystem 1 stored data. Subsystem 2 made LogiTwin think. When a disruption hits any node in the supply chain graph, the engine now detects it, traverses the graph to find every downstream factory affected, scores the impact of each, and writes a scored disruption event back to Neo4j — ready for the mitigation engine in SS3.

The Simulation Pipeline

Step	What Happens	Tool
1. Event arrives	Disruption payload hits POST /simulation/run or /inject/{id}	FastAPI route
2. Task queued	Celery task fired asynchronously — API returns task_id immediately	Celery + Redis
3. Worker picks up	Celery worker dequeues the task from Redis broker	Redis broker
4. Graph traversal	BFS queries find all factories downstream of disrupted node	Neo4j Cypher
5. Impact scoring	Each factory scored: severity × transit_days × (1-capacity)	scorer.py
6. Neo4j write	:DISRUPTION_EVENT node + :IMPACTS relationships created	Neo4j Cypher
7. Result stored	Full simulation result saved to Redis result backend	Redis backend
8. Client polls	GET /simulation/status/{task_id} returns ranked impact report	FastAPI route

02 — 7 New Files Built

File 1	app/workers/celery_app.py	Celery app instance — Redis broker + result backend

- → Single Celery app instance shared across all workers — never instantiated twice
- → Redis used as BOTH the message broker (task queue) and result backend (where results are stored)
- → task_acks_late=True — task only marked complete after successful execution, not just dequeuing
- → worker_concurrency=4 — 4 parallel simulation workers can run simultaneously
- → broker_connection_retry_on_startup=True — worker retries Redis connection on startup

Config: redis://localhost:6379/0 for both broker and backend. result_expires=3600 — results kept 1 hour.

File 2	app/services/traversal.py	BFS graph traversal engine — finds affected factories

Three traversal functions for three disruption types:

- → find_factories_affected_by_port(port_id) — Cypher: MATCH (port)-[:DELIVERS_TO]->(factory). Also walks back to find which suppliers ship through the port. Returns factory + both transit leg data.
- → find_factories_affected_by_supplier(supplier_id) — Runs TWO queries: direct SUPPLIES path + maritime SHIPS_THROUGH→Port→DELIVERS_TO path. Returns unified list with path_type marking each record.
- → find_factory_direct(factory_id) — Returns the factory's own data for direct disruptions (capacity degradation, weather strike).
- → run_traversal(disruption_type, node_id) — Unified dispatcher. Routes to the correct traversal function. Single entry point for all simulation workers.

Verified: All 4 traversal functions import cleanly. Live traversal from port-001 correctly returns fac-001.

File 3	app/services/scorer.py	Impact scoring engine — ranks factories by disruption severity

Scoring Formula

```
impact_score = severity_input × max(total_disrupted_transit_days, 1) × max(1.0 - capacity_rate, 0.05)
Where:
severity_input = 0.0 (minor) to 1.0 (complete failure) — from the disruption event
total_disrupted_transit_days = total supply days lost across the disrupted path
(1 - capacity_rate) = factory vulnerability — lower capacity = higher vulnerability
max(..., 0.05) = floor ensures even 100% capacity factories get a non-zero score
```

Risk Level Thresholds

Risk Level	Impact Score Range	Action Required
CRITICAL	≥ 5.0	Immediate automated mitigation
HIGH	≥ 2.0	Urgent attention — trigger SS3 mitigation
MEDIUM	≥ 0.5	Monitor closely — queue mitigation options
LOW	< 0.5	Log and watch — no immediate action

File 4 **app/workers/simulation.py** Celery task — full simulation pipeline in one task

- → Worker-owned Neo4j driver — Celery workers are separate OS processes from FastAPI. They cannot use the FastAPI lifespan driver. Each task creates its own driver and closes it in a finally block.
- → asyncio.run() wraps the full async pipeline inside the sync Celery task — correct pattern for running async code in Celery workers.
- → Three-step pipeline per task: (1) worker_run_traversal() → (2) score_affected_factories() → (3) write_disruption_event_to_graph()
- → Task state progression: PENDING → STARTED → SUCCESS/FAILURE — pollable via GET /simulation/status/{task_id}
- → SimulationTask base class — on_failure, on_success, on_retry hooks log every lifecycle event with task_id and result summary.
- → max_retries=3 with 5 second countdown — transient Neo4j or Redis failures automatically retried.

Key fix: worker_run_traversal() embeds all Cypher queries directly rather than calling the FastAPI-context-dependent run_traversal() from traversal.py.

File 5 **app/workers/mock_feeds.py** Mock disruption event injectors — 3 realistic scenarios

Scenario	Type	Node	Severity	Description
1	port_congestion	port-001	0.8	JNPT Mumbai congestion — 48hr vessel queues
2	supplier_offline	sup-001	1.0	Tata Components shut down — labour strike
3	capacity_degradation	fac-001	0.9	Delhi Assembly Plant flooded — 30% capacity

File 6 **app/routes/simulation.py** 5 new API endpoints for simulation control

Method	Path	Purpose	Returns
POST	/simulation/run	Fire custom disruption simulation	task_id immediately (202)
GET	/simulation/status/{task_id}	Poll simulation result	Full impact report when done
POST	/simulation/inject/{scenario_id}	Inject mock scenario 1/2/3	task_id immediately (202)
GET	/simulation/scenarios	List available mock scenarios	All 3 scenario definitions
GET	/simulation/events	List all disruption events in Neo4j	Ordered by timestamp desc

File 7 **tests/test_simulation.py** 21 unit tests — all passing in 0.12 seconds

Test Class	Tests	What It Verifies	Result
TestCalculateImpactScore	6	Scoring formula correctness, edge cases, known values	**6/6 PASS**
TestClassifyRiskLevel	4	CRITICAL/HIGH/MEDIUM/LOW threshold boundaries	**4/4 PASS**
TestScoreAffectedFactories	4	Full pipeline: empty results, single, multiple, keys	**4/4 PASS**
TestMockFeeds	7	All scenarios load, invalid raises, shortcut injectors	**7/7 PASS**
TestLiveTraversal	2	Integration: port + supplier traversal against Neo4j	**Run with Neo4j**

03 — Live Simulation Results

All 3 scenarios ran against the live graph and produced correct results.

Scenario	Event ID	Factories Affected	Impact Score	Risk Level	Status
1 — Port Congestion	evt-cea9...	1 (fac-001)	0.96	MEDIUM	pending_mitigation
2 — Supplier Offline	evt-64e7...	2 paths (fac-001)	1.20	HIGH	pending_mitigation
3 — Capacity Degradation	evt-9ff3...	1 (fac-001)	0.135	LOW	pending_mitigation

All 3 :DISRUPTION_EVENT nodes written to Neo4j with :IMPACTS relationships to fac-001. Status 'pending_mitigation' — ready for Subsystem 3 to act on.

04 — Updated Neo4j Graph Schema

Element	Properties	Added in
(:Supplier)	id, name, tier, location	SS1
(:Factory)	id, name, capacity_rate, location	SS1
(:Port)	id, name, average_clearance_days, status	SS1
(:DISRUPTION_EVENT)	id, disruption_type, disrupted_node_id, severity_input, total_factories_affected, highest_risk_level, timestamp, status	**SS2 ✓**
[:SUPPLIES]	component_id, lead_time_days	SS1
[:SHIPS_THROUGH]	transit_days	SS1
[:DELIVERS_TO]	transit_days	SS1
[:IMPACTS]	impact_score, risk_level, path_type	**SS2 ✓**

05 — Startup Commands for Every Session

Run these in order every time you start a new dev session:

Terminal 1 — Start databases

```
docker start logitwin-neo4j logitwin-redis
docker ps | grep -E 'neo4j|redis' # both must show Up
```

Terminal 2 — Start FastAPI

```
cd ~/projects/logitwin && source .venv/bin/activate
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Terminal 3 — Start Celery worker

```
cd ~/projects/logitwin && source .venv/bin/activate
celery -A app.workers.celery_app.celery_app worker --loglevel=info --concurrency=4
```

Terminal 4 — Verify everything live

```
curl http://localhost:8000/health
curl -s -X POST http://localhost:8000/simulation/inject/1 | python3 -m json.tool
```

06 — Subsystem 3 Preview: Automated Mitigation Layer

SS3 picks up every :DISRUPTION_EVENT with status 'pending_mitigation' and automatically finds reroute options, fires carrier API hooks, and logs mitigation actions back to the graph.

- → Hook registry — config-driven map of node IDs to external carrier/supplier API endpoints
- → HTTPX async client pool — per-connector retry logic, timeout management, circuit breaker pattern
- → Mitigation logic — when disruption score exceeds threshold, query graph for alternate SHIPS_THROUGH paths
- → GET /mitigation/options/{disruption_id} — returns ranked reroute candidates with ETA deltas
- → POST /mitigation/execute/{disruption_id} — fires selected hook payload to carrier API
- → Full exception isolation — one failed connector NEVER crashes the mitigation pipeline
- → :MITIGATION_ACTION relationship written to Neo4j for complete audit trail

Resume command for next session:
"Resume LogiTwin — Subsystem 2 complete, begin Subsystem 3: Automated Mitigation Layer"